SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

SCHOOL OF ENGINEERING

ITA06 - MACHINE LEARNING

ASSIGNMENT 3 - CO3

BAYESIAN AND COMPUTATIONAL LEARNING

Student Details	Information
Name	Thikkavarapu Sanhith
Register Number	192311228
Programme / Department	B.E. / B.Tech - Computer Science and Engineering
Course Code	ITA0605
Course Name	Machine Learning
Assignment	Assignment 3 - CO3

Total Marks: 50 | Duration: 1 Week

CONTENTS

- Introduction and experimental protocol
- Q1 - Bayes theorem and concept learning
- Q2 - MDL, Bayes optimal classifier and Gibbs algorithm
- Q3 - Naive Bayes classifier
- Q4 - Bayesian belief network
- Q5 - Expectation-Maximization for Gaussian mixtures
- Q6 - Sample complexity, VC dimension and mistake bounds
- Q7 - Diagnose, redesign and validate
- Overall conclusion, reproducibility and references

INTRODUCTION

Assignment 3 studies how probability and computational learning can be used to make decisions when data are limited, uncertain, incomplete or dependent. The supplied problems cover bearing-fault diagnosis, model selection, loan approval, production-line alarms, unlabelled machine readings and learning-theory guarantees.

The work follows the sequence required by the assignment: predict, model, compute, implement, measure, analyse, diagnose, redesign and validate. The calculations for Q1-Q6 are performed using custom Python implementations. The scikit-learn Naive Bayes implementation is used only as an independent verification for Q3.

Experimental protocol and assumptions

- All random experiments use fixed seeds. The code records seeds and creates CSV result files in the supporting folder.
- Q3 uses a generated 4,000-row binary dataset with a strongly correlated copy of one predictive feature. It is used only for the independence-assumption experiment.
- Q4 simulates 1,000 records from the exact Bayesian network and probabilities given in the assignment. MLE estimates are shown at sample sizes 50, 200 and 1,000.
- Q5 uses the seven readings provided for the manual EM iteration and a separate 500-observation mixture generated from two known Gaussian components.
- Q6 uses a fixed, noise-free target conjunction over ten Boolean features and uniform inputs. Its empirical PAC-style study is illustrative and does not replace the theoretical guarantee.

- Q7 selects the correlated-feature limitation from Q3 and tests a simple feature de-duplication redesign on the same fixed data split.

All synthetic records are generated by the submitted notebook and are not real customer, bearing, machine or student records. Any conclusions from them are limited to the stated generator and assumptions.

Q1 - APPLY BAYES THEOREM AND CONCEPT LEARNING TO HYPOTHESIS SELECTION (5 MARKS)

A. Probability of fault after positive tests

Given: fault prevalence P(F) = 0.03; sensitivity P(+|F) = 0.95; false-positive rate P(+|not F) = 0.08; therefore P(not F) = 0.97.

$$\mathbf{P(F|+) = P(+|F)\,P(F) / [P(+|F)P(F) + P(+|not\ F)P(not\ F)]}$$

First calculate the probability of a positive test:

$$\mathbf{P(+) = (0.95)(0.03) + (0.08)(0.97) = 0.0285 + 0.0776 = 0.1061}$$

$$\mathbf{P(F|+) = 0.0285 / 0.1061 = 0.268615 = 26.8615\%}$$

A positive result does not mean that the bearing is almost certainly faulty. The prior prevalence is only 3%, so the much larger healthy population contributes many false positives. This is the base-rate effect.

For two independent positive tests, independence is assumed conditional on the true fault state. The posterior after the first result becomes the prior for the second result:

$$\mathbf{P(F|++) = (0.2686145)(0.95) / [(0.2686145)(0.95) + (0.7313855)(0.08)]}$$

$$\mathbf{P(F|++) = 0.813478 = 81.3478\%}$$

The direct calculation is 0.03(0.95^2) / [0.03(0.95^2) + 0.97(0.08^2)], which gives the same result. The second independent positive test raises the posterior substantially, but the calculation relies on conditional independence of the two test outcomes.

B. Maintenance records and hypothesis representation

ID	Temperature	Load	Vibration	Fails Early
D1	High	Heavy	High	Yes
D2	High	Heavy	Low	Yes
D3	Normal	Light	Low	No
D4	High	Light	High	No
D5	Normal	Heavy	High	Yes
D6	Normal	Light	High	No

Each hypothesis is a conjunction represented as h = <Temperature, Load, Vibration>. An attribute position can contain a specific domain value or ? for any value. The bottom hypothesis is <empty, empty, empty> and covers no instance; the top hypothesis is <?, ?, ?> and covers every instance.

The complete conjunctive hypothesis space has 3 x 3 x 3 = 27 non-bottom hypotheses: each of the three attributes has its two domain values plus ?. Including the bottom hypothesis gives |H| = 28.

C. Find-S trace

Find-S starts at the most specific hypothesis and uses positive examples only. When a positive instance differs from a constrained value, the relevant constraint becomes ?. Negative examples do not alter Find-S.

Example	Label	S after example	Action
D1	Yes	<High, Heavy, High>	Generalise on positive example
D2	Yes	<High, Heavy, ?>	Generalise on positive example
D3	No	<High, Heavy, ?>	Ignore negative example
D4	No	<High, Heavy, ?>	Ignore negative example
D5	Yes	<?, Heavy, ?>	Generalise on positive example
D6	No	<?, Heavy, ?>	Ignore negative example

The final Find-S hypothesis after D1-D6 is <?, Heavy, ?>. It means Heavy load is the only attribute constraint retained by the positive examples after generalisation.

D. Candidate-Elimination trace

Candidate Elimination maintains S, the maximally specific consistent boundary, and G, the maximally general consistent boundary. Positive examples minimally generalise S and remove any G hypothesis that does not cover the example. Negative examples minimally specialise any G member that covers the negative example, reject specialisations that are not at least as general as S, and remove redundant hypotheses that are less general than another G member.

Step	Label	S boundary	G boundary	Update / reason
D1	Yes	<High, Heavy, High>	<?, ?, ?>	S initialised from the first positive example; G remains top.

Step	Label	S boundary	G boundary	Update / reason
D2	Yes	<High, Heavy, ?>	<?, ?, ?>	Vibration differs, so S generalises its third position to ?.
D3	No	<High, Heavy, ?>	<High, ?, ?>; <?, Heavy, ?>	Negative instance; specialise top G to High temperature or Heavy load.
D4	No	<High, Heavy, ?>	<?, Heavy, ?>	High/Light/High is negative; discard G that covers it and keep the minimally general Heavy-load rule.
D5	Yes	<?, Heavy, ?>	<?, Heavy, ?>	Positive Normal/Heavy/High; generalise S temperature to ?.
D6	No	<?, Heavy, ?>	<?, Heavy, ?>	Negative example is not covered by final G, so boundaries remain unchanged.

For D3, the top hypothesis must be specialised to exclude Normal/Light/Low while retaining both positive examples. The two minimal useful specialisations are <High, ?, ?> and <?, Heavy, ?>. For D4, <High, ?, ?> covers High/Light/High and must be specialised. The specialisation <High, Heavy, ?> is consistent with S, but is removed because <?, Heavy, ?> is more general. The remaining G member already excludes D4.

Final boundaries: S6 = <?, Heavy, ?> and G6 = <?, Heavy, ?>. The boundary is a single hypothesis requiring Heavy load.

E. Version space using D1-D3 and Bayesian hypothesis selection

Using only D1-D3, the two positive examples require a hypothesis to cover High/Heavy/High and High/Heavy/Low. The negative example Normal/Light/Low must not be covered. The three hypotheses satisfying all conditions are:

Hypothesis	Posterior: uniform prior	Posterior: non-uniform prior
<High, Heavy, ?>	0.3333	0.2308
<High, ?, ?>	0.3333	0.4615
<?, Heavy, ?>	0.3333	0.3077

Under a uniform prior and noise-free likelihood, every consistent hypothesis has likelihood 1 and every inconsistent hypothesis has likelihood 0. Thus each of the three version-space members has posterior 1/3. All three are maximum-likelihood hypotheses (hML); there is no unique hML in this dataset.

For a non-uniform prior, assign an unnormalised weight w(h) = 2^(number of ? positions) x 1.5 when Temperature is constrained. This prior favours more general hypotheses, while giving a documented extra preference to rules that retain the measured Temperature attribute. The weights for the three consistent hypotheses are 3, 6 and 4, whose sum is 13. Their posterior probabilities are therefore 3/13 = 0.2308, 6/13 = 0.4615 and 4/13 = 0.3077. The unique MAP hypothesis is <High, ?, ?>.

hML and hMAP coincide when the prior is constant across all hypotheses that maximise the likelihood, or more generally when multiplying the likelihood by the prior does not change the maximiser. A strong prior can resolve a likelihood tie or outweigh a modest likelihood advantage.

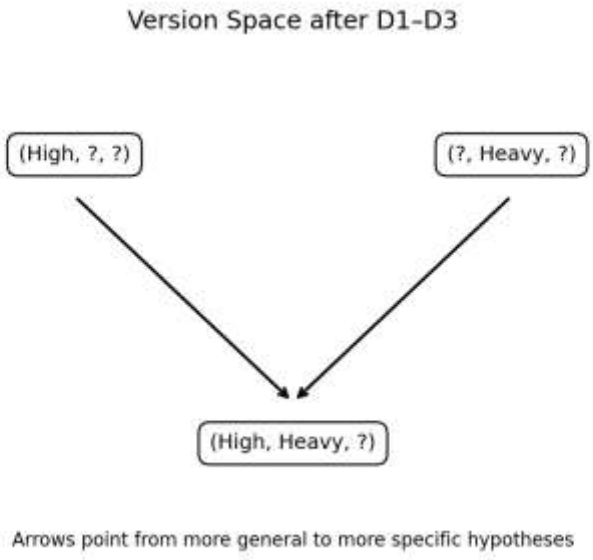


Figure 1. Version-space lattice after D1-D3. Arrows point from more general to more specific hypotheses.

A single MAP hypothesis need not agree with the posterior-weighted vote. For query <High, Light, High>, the MAP hypothesis <High, ?, ?> predicts Yes. The other two version-space hypotheses predict No. The total posterior probability of Yes is only 0.4615, so Bayesian model averaging predicts No. This illustrates why using the whole posterior can differ from choosing one best hypothesis.

Q2 - COMPARE MDL, BAYES OPTIMAL CLASSIFIER AND GIBBS ALGORITHM (5 MARKS)

A. Minimum Description Length calculations

The MDL score is the number of bits used to describe the model plus the number of bits needed to encode its errors. Given 200 records, an error costs log2(200) = 7.643856 bits. Each tree node costs 12 bits.

L(Tree) = (number of nodes x 12) + (number of errors x log2(200))

Tree	Model bits	Error bits	Total description length
A: 6 nodes, 8 errors	6 x 12 = 72	8 x 7.643856 = 61.150849	133.150849 bits
B: 25 nodes, 1 error	25 x 12 = 300	1 x 7.643856 = 7.643856	307.643856 bits

MDL selects Tree A because 133.150849 bits is less than 307.643856 bits. Tree B makes fewer errors, but its larger description cost dominates at the stated 12-bit node cost.

Let c be the cost per node. The reversal occurs when 6c + 8log2(200) = 25c + log2(200). Therefore 19c = 7log2(200), and c = 2.816158 bits per node. For c below this value, Tree B is preferred; above it, Tree A is preferred. At c = 12, Tree A is preferred.

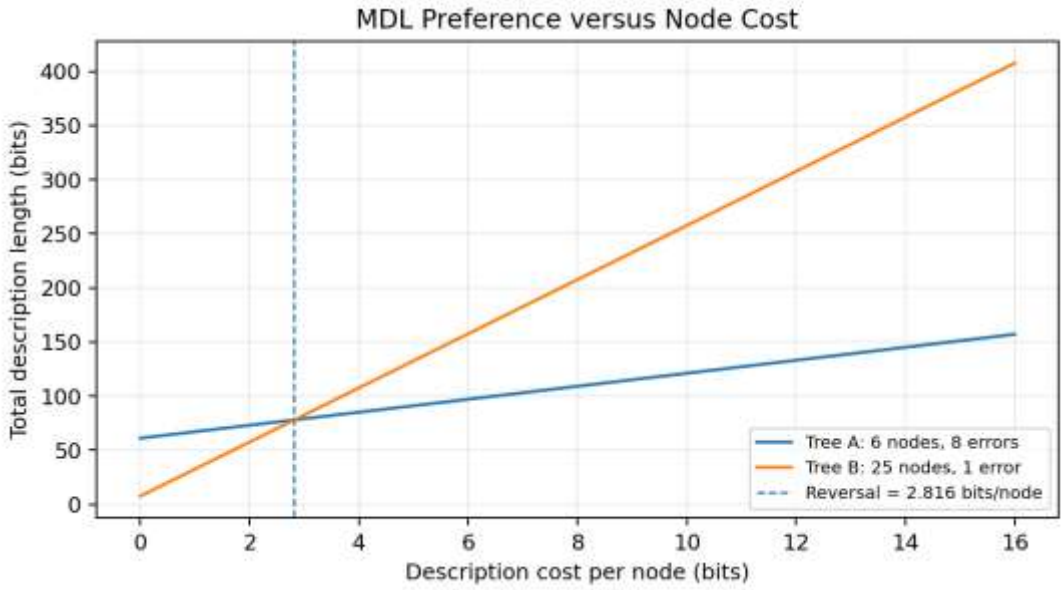


Figure 2. MDL score as the description cost per node changes.

The link with MAP follows from coding theory: a shorter code length corresponds to a higher probability. Up to coding constants, L(h) = -log2 P(h) and L(D|h) = -log2 P(D|h). Minimising their sum is equivalent to maximising P(h)P(D|h), which is the MAP rule.

B. MAP and Bayes-optimal prediction

Hypothesis	Posterior P(h|D)	Prediction for X	Contribution
h1	0.30	+	0.30 to +
h2	0.25	-	0.25 to -
h3	0.20	-	0.20 to -
h4	0.15	+	0.15 to +
h5	0.10	-	0.10 to -
Total +	0.45		
Total -	0.55		

The MAP hypothesis is h1 because it has the largest individual posterior probability, 0.30. It predicts +. The Bayes-optimal classifier sums posterior mass by class: P(+|D) = 0.30 + 0.15 = 0.45 and P(-|D) = 0.25 + 0.20 + 0.10 = 0.55. Therefore the Bayes-optimal prediction is -, which differs from the MAP hypothesis prediction.

C. Gibbs algorithm and expected error

The Gibbs algorithm samples one hypothesis from the posterior distribution and uses that sampled hypothesis for prediction. When the true hypothesis is also assumed to be drawn independently from the posterior, the sampled hypothesis disagrees with the true hypothesis on X when one predicts + and the other predicts -.

$$\textbf{P(Gibbs error) = P(+)P(-) + P(-)P(+) = 2(0.45)(0.55) = 0.495}$$

The Bayes-optimal classifier predicts the posterior-majority label, so its expected error on X is the minority posterior mass: min(0.45, 0.55) = 0.45. The relation is 0.45 <= 0.495 <= 0.90 = 2 x 0.45. The Bayes-optimal classifier has the lower expected 0-1 error.

A Monte Carlo simulation with 200,000 independent pairs of posterior draws and seed 4202 produced a Gibbs disagreement rate of 0.494505, close to the analytical 0.495. The difference is ordinary finite-simulation variation.

The Bayes-optimal prediction at each point is a posterior majority vote. Across all possible inputs, the function formed by combining these votes need not be representable by a single hypothesis in H. Exact Bayesian averaging can also be expensive when H is very large because many posterior hypotheses must be summed. Gibbs offers a practical approximation by sampling one posterior hypothesis rather than explicitly averaging all of them.

Q3 - BUILD AND EVALUATE A NAIVE BAYES CLASSIFIER (6 MARKS)

A. Assumption, decision rule and class priors

Naive Bayes assumes that the features are conditionally independent given the class. For Income, Employment and CreditHistory:

$$P(X|Approve=c) = P(Income|c)\ P(Employment|c)\ P(CreditHistory|c)$$

The decision rule selects the class with the largest posterior, equivalently the largest product of prior and conditional likelihoods. The ten supplied loan records contain five Yes and five No decisions, so P(Yes) = 0.5 and P(No) = 0.5.

B. Conditional probability tables

Attribute	Value	P(value | Yes)	P(value | No)
Income	Low	0.2000	0.6000
Income	Medium	0.4000	0.2000
Income	High	0.4000	0.2000
Employment	Salaried	0.8000	0.2000
Employment	Self-employed	0.2000	0.8000
CreditHistory	Good	0.8000	0.2000
CreditHistory	Poor	0.2000	0.8000

These maximum-likelihood estimates use counts divided by five records in each class. For example, four of the five approved applications have Good credit history, so P(Good|Yes) = 4/5 = 0.8. One of the five rejected applications has Good credit history, so P(Good|No) = 1/5 = 0.2.

C. Manual classifications

Application X1 = (Low, Self-employed, Poor):

$$\textbf{Score(Yes)} = 0.5 \times 0.2 \times 0.2 \times 0.2 = 0.004$$

$$\textbf{Score(No)} = 0.5 \times 0.6 \times 0.8 \times 0.8 = 0.192$$

$$P(Yes|X1) = 0.004 / 0.196 = 0.020408;\ \ P(No|X1) = 0.979592$$

Prediction: No.

Application X2 = (Medium, Salaried, Good):

Score(Yes) = 0.5 x 0.4 x 0.8 x 0.8 = 0.128

Score(No) = 0.5 x 0.2 x 0.2 x 0.2 = 0.004

P(Yes|X2) = 0.128 / 0.132 = 0.969697; P(No|X2) = 0.030303

Prediction: Yes.

D. Unseen income and Laplace smoothing

The value Very High was not observed in training. Under the maximum-likelihood estimate, P(Very High|Yes) = P(Very High|No) = 0. Therefore both unnormalised class scores for (Very High, Salaried, Good) become zero and the posterior cannot be normalised.

Use Laplace smoothing with alpha = 1. Income has four declared categories (Low, Medium, High, Very High); Employment and CreditHistory each have two categories. The smoothed conditional probability is:

P(value|class) = (count(value,class) + alpha) / (N_class + alpha x K)

Probability	Smoothed estimate
P(Very High|Yes) and P(Very High|No)	1 / (5 + 1 x 4) = 1/9 = 0.1111
P(Salaried|Yes)	(4+1)/(5+2) = 5/7 = 0.7143
P(Salaried|No)	(1+1)/(5+2) = 2/7 = 0.2857
P(Good|Yes)	(4+1)/(5+2) = 5/7 = 0.7143
P(Good|No)	(1+1)/(5+2) = 2/7 = 0.2857

Smoothed score(Yes) = 0.5 x (1/9) x (5/7) x (5/7) = 0.0283447

Smoothed score(No) = 0.5 x (1/9) x (2/7) x (2/7) = 0.0045351

P(Yes|X) = 0.862069; P(No|X) = 0.137931

With alpha = 1, the smoothed prediction is Yes. Smoothing prevents unseen categories from forcing a zero likelihood. The same alpha and category vocabulary are used for all categorical features in this demonstration.

E. From-scratch implementation and library verification

The notebook implements class priors, conditional tables, log-probability scoring and posterior normalisation directly. In log space, products become sums: log P(c|x) is proportional to log P(c) + sum_j log P(x_j|c). This reduces numerical underflow for long feature vectors and makes zero-probability events explicit as negative infinity.

For the two known manual applications, the scratch implementation with alpha = 0 produced the same posteriors as scikit-learn CategoricalNB(alpha=0): X1 gives Yes = 0.020408 and No = 0.979592; X2 gives Yes = 0.969697 and No = 0.030303. For the unseen query, both the scratch and scikit-learn alpha = 1 implementations give Yes = 0.862069 and No = 0.137931.

F. Correlated or duplicated-feature experiment

A reproducible binary dataset of 4,000 records was generated with seed 3303 and exported as q3_correlated_features_4000.csv. X1 is predictive of the target with 82% agreement; X2 is a 97%-accurate noisy copy of X1; X3 is an independent, weaker signal with 60% agreement. The first 2,800 shuffled records are used for training and the remaining 1,200 for testing. A fixed 70/30 split is used for both models.

Naive Bayes model	Test accuracy	Test log-loss
Without redundant X2	0.8275	0.453632
With correlated X2	0.8225	0.551920

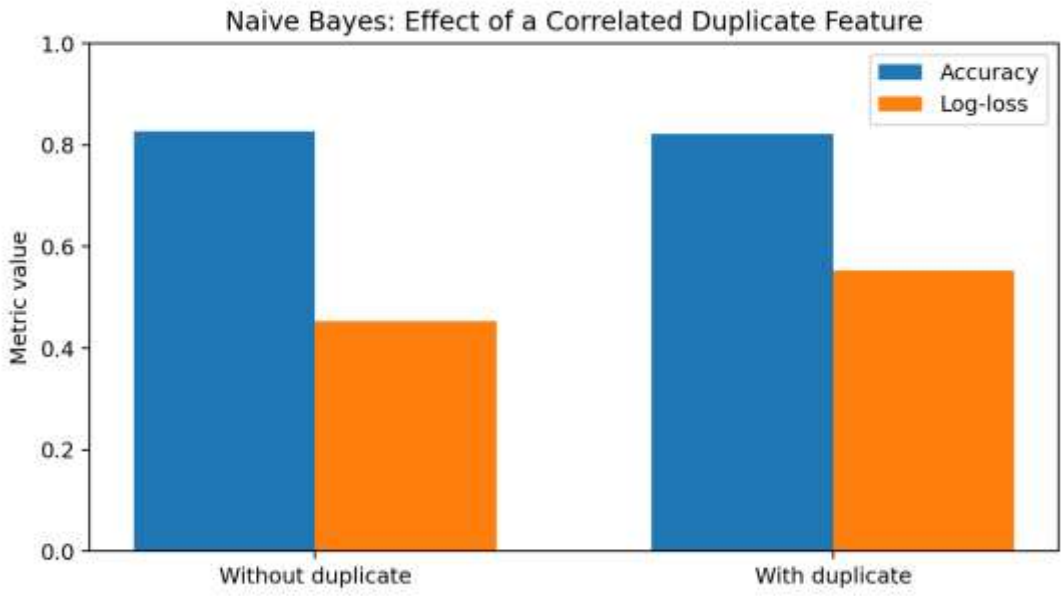


Figure 3. Accuracy and log-loss with and without the correlated duplicate feature.

Including the redundant feature reduced accuracy by 0.5 percentage points and increased log-loss by 0.098289 (about 17.81% relative to the redundant-feature model). Thus, a violated conditional-independence assumption does not always cause a large change in classification accuracy, but it can make probabilities less reliable by counting closely related evidence more than once. Log-loss reveals this calibration/confidence problem more clearly than accuracy alone.

Q4 - MODEL AND QUERY A BAYESIAN BELIEF NETWORK (6 MARKS)

A. Network structure and factorisation

The five binary variables are P (Power Surge), C (Coolant Failure), O (Machine Overheating), A (Sensor Alarm) and S (Line Shutdown). P and C are independent root causes. Both point to overheating; overheating points to the alarm and shutdown.

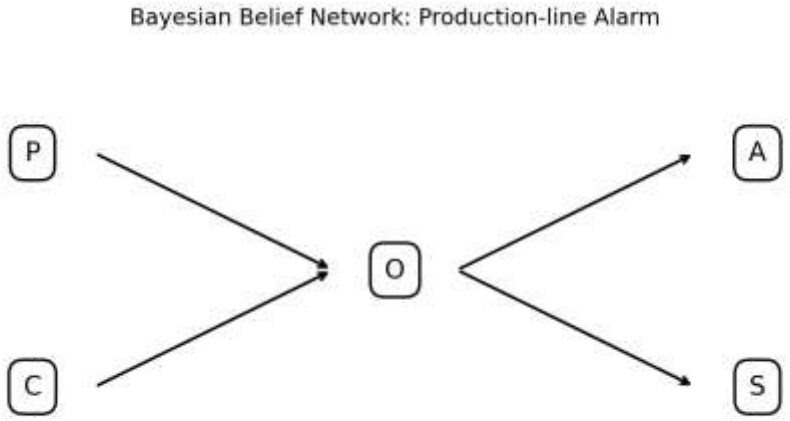


Figure 4. Directed acyclic graph for the production-line alarm system.

P(P,C,O,A,S) = P(P) P(C) P(O|P,C) P(A|O) P(S|O)

CPT	Parameters
P(P)	1
P(C)	1
P(O|P,C)	4
P(A|O)	2
P(S|O)	2
Total Bayesian network	10
Full joint over 5 binary variables	2^5 - 1 = 31

A binary variable with no parents needs one independent probability. A binary child with k binary parents needs 2^k probabilities, one for each parent configuration. The network therefore needs 10

independent parameters, compared with 31 for an unrestricted joint distribution over five binary variables.

B. Conditional-independence and d-separation

Claim	Holds?	Reason
A independent of S given O	Yes	A and S are separate children of O; conditioning on O blocks their common-cause path.
P independent of C with no evidence	Yes	P and C are independent root variables in the factorisation.
P independent of C given O	No	O is a common effect (collider); observing it opens an explaining-away association.
P independent of C given A	No	A is a descendant of collider O; observing the descendant also opens the path.

C. Manual inference

The probability that overheating is true is the sum over the four P,C configurations:

P	C	P(P)P(C)	P(O=T|P,C)	Joint contribution
T	T	0.10 x 0.05 = 0.005	0.95	0.00475
T	F	0.10 x 0.95 = 0.095	0.60	0.05700
F	T	0.90 x 0.05 = 0.045	0.80	0.03600
F	F	0.90 x 0.95 = 0.855	0.02	0.01710
				Total = 0.11485

$$\mathbf{P(O=T) = 0.00475 + 0.05700 + 0.03600 + 0.01710 = 0.11485}$$

For the query P(C=T|A=T), first calculate P(A=T) = P(O=T)P(A=T|O=T) + P(O=F)P(A=T|O=F):

$$\mathbf{P(A=T) = (0.11485)(0.90) + (0.88515)(0.05) = 0.1476225}$$

The joint probability P(C=T,A=T) is the sum over both values of P:

$$\mathbf{P(C=T,A=T) = 0.05[0.10(0.95 \text{ x } 0.90 + 0.05 \text{ x } 0.05) + 0.90(0.80 \text{ x } 0.90 + 0.20 \text{ x } 0.05)]}$$

P(C=T,A=T) = 0.0371375

P(C=T|A=T) = 0.0371375 / 0.1476225 = 0.251571

When Power Surge is also known to be true:

P(C=T,A=T,P=T) = 0.10 x 0.05 x (0.95 x 0.90 + 0.05 x 0.05) = 0.0042875

P(A=T,P=T) = 0.10[0.05(0.8575) + 0.95(0.60 x 0.90 + 0.40 x 0.05)] = 0.0574875

P(C=T|A=T,P=T) = 0.0042875 / 0.0574875 = 0.074581

This is explaining away. Before observing P, an alarm raises the probability of coolant failure to 25.16%. Once a power surge is known, it already explains much of the alarm risk, so the conditional probability of coolant failure falls to 7.46%.

D. Enumeration implementation and term counts

The custom inference enumerates all 2^5 = 32 complete assignments and multiplies the local conditional probabilities. The code sums assignments that satisfy the query and divides by the sum satisfying the evidence.

Query	Numerator terms	Denominator terms	Result
P(O=T)	16	32	0.114850
P(C=T|A=T)	8	16	0.251571
P(C=T|A=T,P=T)	4	8	0.074581

The enumeration counts are the number of full joint assignments satisfying the numerator and evidence conditions. Exact enumeration is simple for five variables, but the number of assignments grows exponentially with the number of binary variables.

E. MLE estimation from simulated samples

A fixed seed (4404) generated 1,000 rows from the supplied network. All root probabilities and conditional tables were estimated by relative frequencies at sample sizes 50, 200 and 1,000. For P(O|P,C), the mean absolute error (MAE) is computed over the parent configurations observed in the sample.

N	P(P)	P(C)	O|00	O|01	O|10	O|11	O-CPT MAE

N	P(P)	P(C)	O|00	O|01	O|10	O|11	O-CPT MAE
50	0.080	0.100	0.000	0.800	0.750	NA	0.0567
200	0.095	0.070	0.012	0.769	0.500	1.000	0.0472
1000	0.106	0.044	0.020	0.769	0.614	0.800	0.0487

At N = 50, the parent configuration P=T,C=T did not occur, so its maximum-likelihood conditional estimate is unavailable (NA). The MAE for that row is averaged over the three observed parent configurations only. The rare P=T,C=T case remains poorly estimated even at N=1,000 because the root events are uncommon. The experiment demonstrates why conditional tables with rare parent configurations require enough observations or smoothing/structural assumptions.

F. Comparison with Naive Bayes

Naive Bayes imposes conditional independence among all input features once the class is known. This belief network instead represents explicit causal/dependency links: P and C jointly influence O, and O influences A and S. The additional structure can represent dependencies that Naive Bayes cannot. The cost is more complex inference; exact enumeration over n binary variables requires considering 2^n complete assignments in the straightforward method.

Q5 - DEVELOP AND ANALYSE THE EM ALGORITHM FOR A MIXTURE MODEL (7 MARKS)

A. Gaussian mixture model and latent variables

The seven supplied readings are 1.8, 2.5, 3.1, 5.0, 7.6, 8.4 and 9.0. Each reading comes from one of two Gaussian-producing machines, but the machine identity is hidden. Let z_i indicate the unknown component assignment.

$$p(x_i) = \sum_{k=1..K} \pi_k \, N(x_i \mid \mu_k, \sigma_k^2)$$

The mixing proportions satisfy $\pi_k \geq 0$ and $\sum \pi_k = 1$. The hidden variable z_i is a one-of-K component indicator. The E-step calculates the responsibility of component k for observation i:

$$\gamma_{ik} = \pi_k \, N(x_i|\mu_k,\sigma_k^2) \, / \, \sum_j \pi_j \, N(x_i|\mu_j,\sigma_j^2)$$

Let $N_k = \sum_i \gamma_{ik}$. The M-step updates:

$$\pi_k(new) = N_k / N$$

$$\mu_k(new) = \sum_i \gamma_{ik} \, x_i / N_k$$

$$\sigma_k^2(new) = \sum_i \gamma_{ik} \, (x_i - \mu_k(new))^2 / N_k$$

B. Derivation of the mean update

The expected complete-data log-likelihood contains the Gaussian quadratic term: $Q(\mu_k) = \sum_i \gamma_{ik}[-(x_i-\mu_k)^2/(2\sigma_k^2)]$ plus terms that do not depend on μ_k. Differentiating and setting the result to zero gives:

$$dQ/d\mu_k = \sum_i \gamma_{ik} \, (x_i - \mu_k) / \sigma_k^2 = 0$$

$$\mu_k \sum_i \gamma_{ik} = \sum_i \gamma_{ik} \, x_i$$

$$\mu_k(new) = \sum_i \gamma_{ik} \, x_i / \sum_i \gamma_{ik}$$

Thus the updated mean is the responsibility-weighted average of the observations.

C. One manual EM iteration on the seven readings

Initial parameters are $\pi_1 = \pi_2 = 0.5$, $\mu_1 = 2$, $\mu_2 = 8$, and $\sigma_1^2 = \sigma_2^2 = 1$. The E-step responsibilities are:

Reading x	Responsibility for component 1	Responsibility for component 2

Reading x	Responsibility for component 1	Responsibility for component 2
1.8	1.00000000	0.00000000
2.5	0.99999969	0.00000031
3.1	0.99998880	0.00001120
5.0	0.50000000	0.50000000
7.6	0.00000017	0.99999983
8.4	0.00000000	1.00000000
9.0	0.00000000	1.00000000

The seven responsibilities are then summed for each component and used in the M-step. After one iteration:

pi1 = 0.4999984; pi2 = 0.5000016

mu1 = 2.8285708; mu2 = 7.8571272

sigma1^2 = 1.0277593; sigma2^2 = 1.6425186

The observed-data log-likelihood increases from -16.501441 before the iteration to -14.984221 after it. The increase is consistent with the EM monotonicity property for an exact E-step and M-step.

D. Reproducible 500-observation experiment

A synthetic one-dimensional dataset of 500 readings is generated using seed 5505. The hidden component is sampled with probabilities 0.55 and 0.45. Component 1 has N(2.0, 0.55^2); component 2 has N(7.0, 1.0^2). The component identity is retained only for generator documentation and is not used during EM fitting.

The best two-component fit across ten random initialisations has mixing weights about 0.5455 and 0.4545, means 2.0007 and 6.8998, variances 0.3373 and 1.1112, and log-likelihood -914.410451. The values are close to the data-generating parameters, subject to sampling variation.

E. Ten initialisations, local optima and label switching

Run	Seed	Final log-likelihood	Iterations	mu1	mu2

Run	Seed	Final log-likelihood	Iterations	mu1	mu2
1	5600	-914.410451	43	2.0007	6.8997
2	5601	-914.410451	21	2.0007	6.8998
3	5602	-914.410451	14	2.0007	6.8997
4	5603	-914.410451	16	2.0007	6.8998
5	5604	-914.410451	14	2.0007	6.8998
6	5605	-914.410451	22	2.0007	6.8997
7	5606	-914.410451	47	2.0007	6.8998
8	5607	-914.410451	20	2.0007	6.8997
9	5608	-914.410451	32	2.0007	6.8997
10	5609	-914.410451	15	2.0007	6.8997

All ten starts converged to essentially the same final log-likelihood (approximately -914.410451); no distinct inferior local optimum was observed among these ten starts. This does not prove that EM has no local optima. It only reports what happened for this well-separated generated dataset and these initialisations.

Label switching was demonstrated by fitting the same data with initial means [2, 7] and [7, 2]. The resulting component parameters appear in reversed order, but the final log-likelihood is the same (-914.410451). Component labels are arbitrary; the mixture distribution is unchanged by swapping component indices.

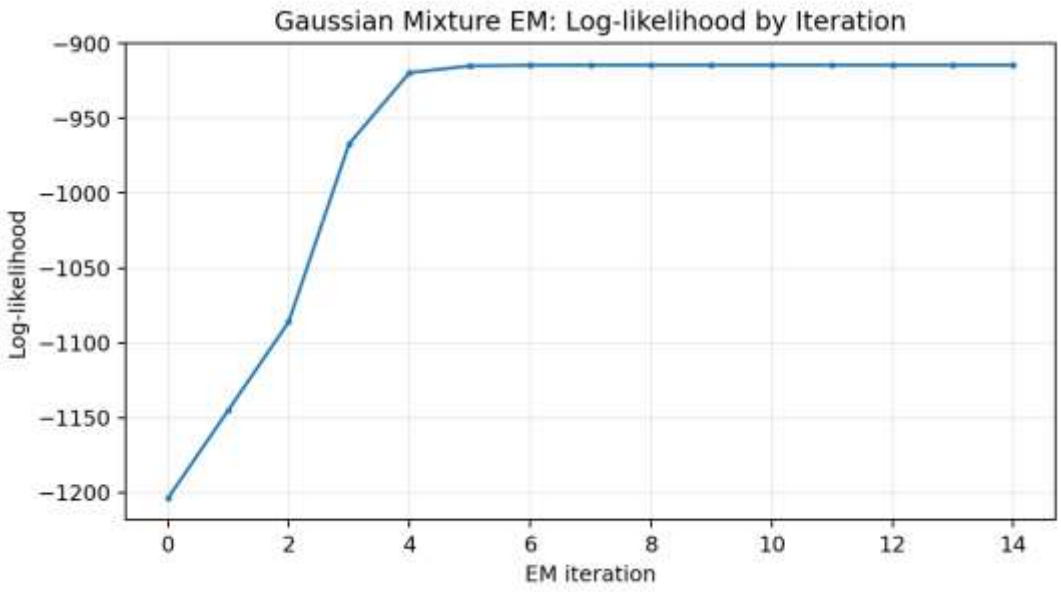


Figure 5. Observed-data log-likelihood across EM iterations for the selected two-component fit.

F. Choosing the number of components with BIC

Components K	Log-likelihood	Free parameters	BIC	Iterations
1	-1182.728	2	2377.886	2
2	-914.410	5	1859.894	9
3	-912.580	8	1874.878	500

For a univariate K-component Gaussian mixture, the parameter count is (K-1) mixing weights plus K means and K variances, giving 3K-1 parameters. BIC = -2 log-likelihood + p ln(N). K=2 has the smallest BIC (1859.894), so it is preferred among the tested models. K=3 has a slightly higher likelihood but a larger BIC; its run also reached the 500-iteration cap, so the K=3 comparison should be interpreted cautiously.

G. Soft EM versus hard assignment

Soft EM achieved log-likelihood -914.410451, with means [2.0007112538267617, 6.89975046788007] and variances [0.33728126475262, 1.1112000000766418]. The hard-assignment (k-means-style) procedure achieved log-likelihood -914.593253, with means [2.0084207320239376, 6.917530047012877] and variances [0.34884948127927146, 1.0597598321286057]. Soft EM is slightly

better on this dataset because it retains uncertainty about component membership rather than assigning each reading completely to one machine at every iteration.

Each exact EM iteration does not decrease observed-data likelihood because the E-step forms the expected complete-data objective using the current posterior over hidden variables, and the M-step maximises that objective. The full iteration-by-iteration values are exported in q5_em_loglik_history.csv; the smallest recorded increment is approximately 1.16 x 10^-8, consistent with numerical convergence. Monotonic improvement does not guarantee a global maximum: EM is a local optimisation method and can converge to a local stationary point. Multiple initialisations reduce sensitivity but do not prove global optimality.

The same idea can estimate a Bayesian-network CPT with hidden variables: the E-step computes expected counts for unobserved parent/child assignments, and the M-step normalises those expected counts into conditional probabilities.

Q6 - SAMPLE COMPLEXITY, HYPOTHESIS SPACE CAPACITY AND MISTAKE BOUNDS (8 MARKS)

A. PAC learning and finite hypothesis spaces

PAC learning expresses a probabilistic generalisation guarantee. With probability at least 1-delta over the training sample, the learned hypothesis has true error at most epsilon. Smaller epsilon demands more accuracy; smaller delta demands higher confidence.

$$m >= (1/\text{epsilon}) [\ln|H| + \ln(1/\text{delta})]$$

For conjunctions of literals over n Boolean variables, the assignment specifies $|H| = 3^n + 1$. At $n = 10$, $|H| = 3^{10} + 1 = 59{,}050$.

epsilon	delta	Raw bound m	Required integer m
0.10	0.05	139.819	140
0.10	0.01	155.913	156
0.05	0.05	279.637	280
0.05	0.01	311.826	312

For epsilon = 0.10 and delta = 0.05: ln(59,050) + ln(20) = 13.981872, and m >= 139.8187, so m = 140. Halving epsilon from 0.10 to 0.05 doubles the bound. Reducing delta from 0.05 to 0.01 increases the

bound more slowly because delta appears inside a logarithm. In this formula epsilon has the strongest direct influence because of the 1/epsilon factor; the logarithm of the hypothesis-space size grows approximately linearly with n.

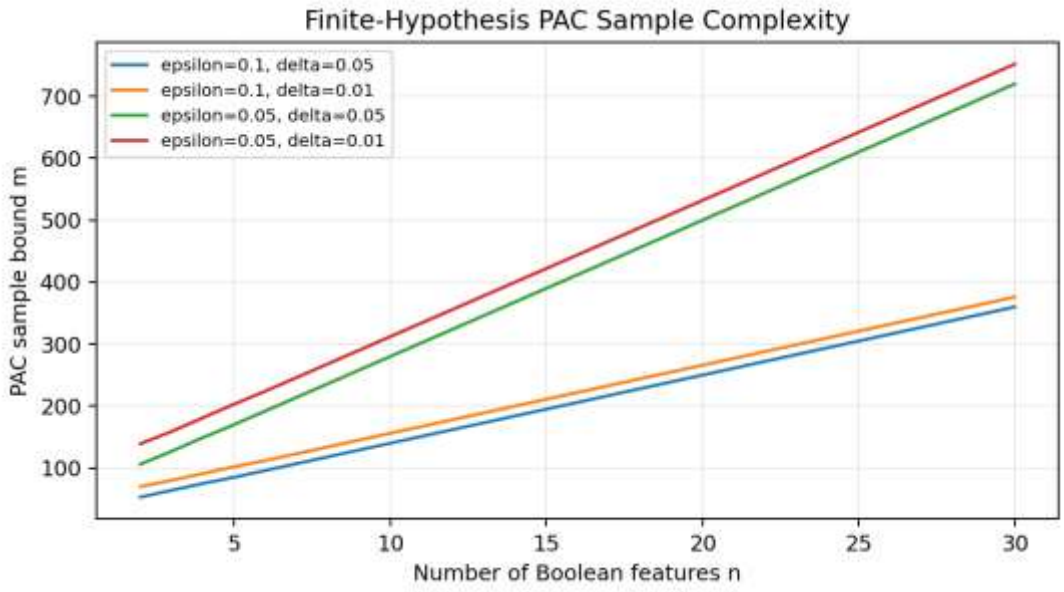


Figure 6. Finite-hypothesis PAC sample bound as feature count, epsilon and delta vary.

B. Find-S simulation against the true target

The fixed noise-free target is the conjunction X1=1 AND X2=1 AND X3=0 over ten Boolean features. Inputs are sampled uniformly from the 10-dimensional Boolean cube. Find-S is trained on 200 independent trials per sample size. The true error is measured exactly by evaluating the learned hypothesis on all 2^10 = 1,024 possible inputs.

Training m	Mean true error	Fraction with error <= 0.10	Fraction with error <= 0.05
10	0.29473	0.120	0.015
20	0.15479	0.440	0.140
40	0.04452	0.860	0.505
80	0.00352	0.995	0.950
120	0.00094	1.000	0.985
140	0.00000	1.000	1.000
156	0.00000	1.000	1.000

Training m	Mean true error	Fraction with error <= 0.10	Fraction with error <= 0.05
200	0.00000	1.000	1.000
280	0.00000	1.000	1.000
300	0.00000	1.000	1.000
312	0.00000	1.000	1.000

At the four finite-space bounds, the simulation has success fractions 1.000 for epsilon=0.10 at m=140 and m=156, and 1.000 for epsilon=0.05 at m=280 and m=312. These exceed the required 0.95 or 0.99 confidence fractions in the 200 trials. The bound is conservative for this target and uniform distribution: the simulation already reaches high success at smaller m. The experiment is specific to this target and distribution; it is not a proof of the PAC theorem.

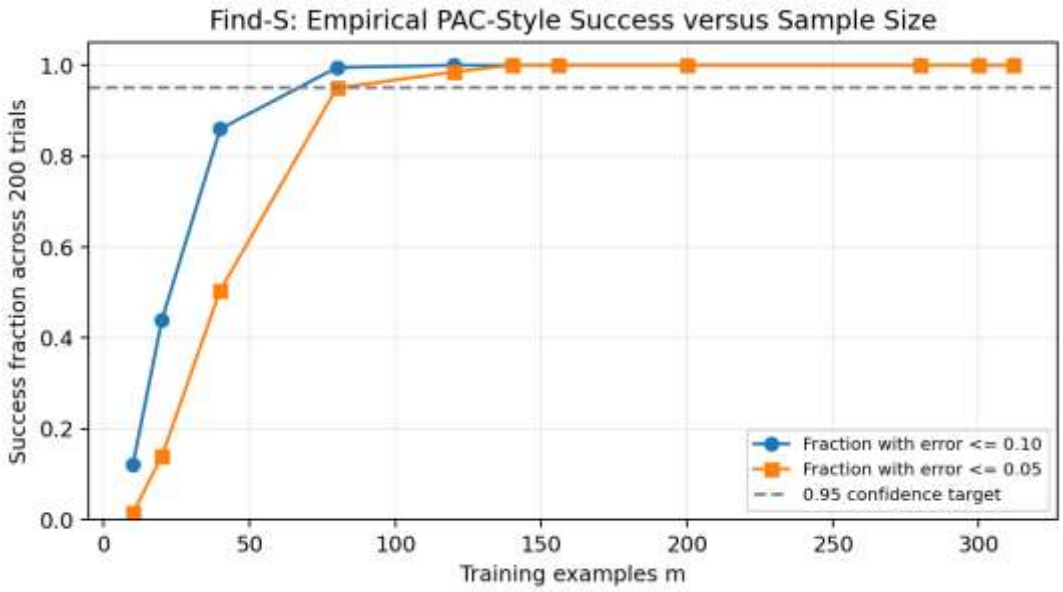


Figure 7. Empirical fraction of Find-S trials meeting each error threshold.

C. VC dimension and shattering

A set of points is shattered by a hypothesis class if every possible binary labelling of those points can be produced by some hypothesis in the class.

Hypothesis class	Explicit lower-bound argument	Upper-bound argument	VC dimension
Intervals on the real line	Any two ordered points can realise all four labellings using an empty interval, a small interval around either point, or one interval covering both.	Three ordered points cannot realise positive, negative, positive because one interval containing the outer points must also contain the middle point.	2
Linear separators in the plane	Three non-collinear points can realise all eight labelings with a line or half-plane.	Four points in convex position with alternating diagonal labels cannot be separated by one line.	3
Axis-aligned rectangles in the plane	Four points in a cross configuration (top, right, bottom, left) can be isolated in every subset by choosing rectangle boundaries.	Among five points, at least one is not a unique coordinate extreme; a rectangle containing the four extreme points also contains that interior point, preventing one labelling.	4

Using the assignment-provided VC bound with epsilon = 0.1 and delta = 0.05 gives:

m >= (1/epsilon)[4 log2(2/delta) + 8 VC(H) log2(13/epsilon)]

Class	VC dimension	Raw bound	Required integer m
Intervals on real line	2	1336.456	1337
Linear separators in plane	3	1898.245	1899
Axis-aligned rectangles in plane	4	2460.035	2461

The finite-space bound depends on ln|H|. The VC bound depends on the expressive capacity of an infinite hypothesis class through VC dimension. Both are sufficient sample bounds and can be conservative; their numerical values come from different formulas and log conventions.

D. Mistake-bound model: Halving algorithm

Use a finite hypothesis space of 64 hypotheses, each a six-bit label vector over six input points. The Halving algorithm predicts the majority label among currently consistent hypotheses and removes every hypothesis inconsistent with the revealed label. The mistake bound is log2(64) = 6.

Step	H before	Votes 0	Votes 1	Prediction	True label	Mistake
1	64	32	32	0	1	1
2	32	16	16	0	1	1
3	16	8	8	0	1	1
4	8	4	4	0	1	1
5	4	2	2	0	1	1
6	2	1	1	0	1	1

The fixed tie-breaking rule predicts 0 when votes are tied. The adversarial realizable sequence labels each queried point 1. The consistent set halves from 64 to 32, 16, 8, 4, 2 and finally 1, so the learner makes six mistakes and reaches the bound log2|H| exactly.

E. Online learner for conjunctions

The online conjunction learner begins at the most-specific bottom hypothesis, which predicts 0. On the first positive example it installs the literals matching all feature values. On each later positive example it removes any literal contradicted by that positive instance. Under a realizable conjunction target, all target literals remain in the current hypothesis; therefore a negative example cannot be falsely accepted. A mistake can occur only when a positive example violates an extra literal, and that mistake removes at least one extra literal. There can be at most n such removals, plus the initial positive-example mistake, giving at most n+1 mistakes.

Across 500 random noise-free sequences with n = 10, the observed maximum was 7 mistakes and the mean was 2.556. The theoretical bound is n+1 = 11, so all simulated runs respected it.

F. Relation between VC dimension and optimal mistake bound

For thresholds on the ordered domain {1,2,3}, there are four threshold concepts (the positive region can end before the first point, after the first, after the second or after the third). The class has VC dimension 1, and online binary search can require at most two mistakes in the worst case. Therefore:

$$\mathbf{VC(C) = 1 <= Opt(C) = 2 <= log2|C| = log2(4) = 2}$$

The PAC framework asks how many labelled examples are sufficient to guarantee low generalisation error with high probability. A mistake-bound model asks how many prediction errors an online learner can make over a sequence. PAC guarantees are useful for offline quality-model deployment planning; mistake bounds are useful when labelled inspections arrive sequentially and the learner updates after each case. Both rely on assumptions such as a realisable target for the stated guarantees; label noise can invalidate the clean bounds.

Q7 - DIAGNOSE, REDESIGN AND VALIDATE A BAYESIAN OR COMPUTATIONAL LEARNING SOLUTION (13 MARKS)

Selected limitation: conditional independence violated by a redundant feature

The selected solution is the Naive Bayes experiment in Q3. The generator deliberately creates X2 as a 97%-accurate noisy copy of X1. Since X1 and X2 measure nearly the same underlying signal, they are strongly dependent even after conditioning on the class. Naive Bayes nevertheless multiplies both conditional likelihoods as if they were independent. This double-counts correlated evidence and makes posterior probabilities too confident.

A. Diagnose the learning bottleneck (2 marks)

The baseline with both X1 and redundant X2 achieves test accuracy 0.8225 and log-loss 0.551920. The main limitation is a model-assumption problem rather than insufficient sample size: the conditional-independence assumption is violated by construction. Accuracy changes only slightly, but log-loss becomes worse, indicating less reliable probability estimates.

B. Define the design objective (2 marks)

Before redesign, set the measurable objective as: reduce held-out log-loss by at least 10% relative to the two-feature-dependence baseline while keeping accuracy no more than 2 percentage points below the baseline. Use the same generated data, seed, train/test indices, smoothing and evaluation metrics.

C. Revised learning strategy (4 marks)

The prototype removes the redundant copy X2 and retains X1 plus the independent weak signal X3. The class priors and conditional probabilities are refitted on the same training split. This reduces duplicated evidence without changing the Naive Bayes implementation or using additional model parameters.

Revised workflow: (1) measure pairwise feature dependence on the training data; (2) identify exact or near-duplicate predictors; (3) retain one representative from each redundant group or use a dependency-aware model when the relationship is useful; (4) fit Naive Bayes on the reduced feature set; (5) evaluate log-loss and accuracy on a held-out set. A more expressive alternative is Tree-Augmented Naive Bayes (TAN), which permits one additional parent feature for each attribute, but it needs additional structure learning and more data.

Pseudocode:

- INPUT: training data D, features X, target Y, validation data V
- Compute pairwise dependence scores using training data only

- Identify redundant pair (X1, X2) and select representative X1
- X_reduced <- {X1, X3}
- Fit class priors and smoothed conditional tables on D using X_reduced
- Predict posterior probabilities for V in log space
- Compare validation log-loss and accuracy with the baseline
- ACCEPT redesign only if objective thresholds are met

The redesign requires little extra memory and can reduce computation because one feature is removed. Its limitation is that a feature that is correlated but contains unique information should not be removed automatically. Feature dependence must be checked using training data and domain knowledge.

D. Prediction before implementation (1 mark)

Prediction recorded before validation: removing X2 should reduce overconfidence and improve log-loss, while accuracy should remain similar because X2 is mostly a copy of X1. The targeted measures are log-loss and accuracy change.

E. Implement and validate (2 marks)

Measure	Baseline: with X2 duplicate	Redesign: without X2	Change
Test accuracy	0.8225	0.8275	+0.0050 = +0.50 percentage points
Test log-loss	0.551920	0.453632	-0.098289 = 17.81% reduction

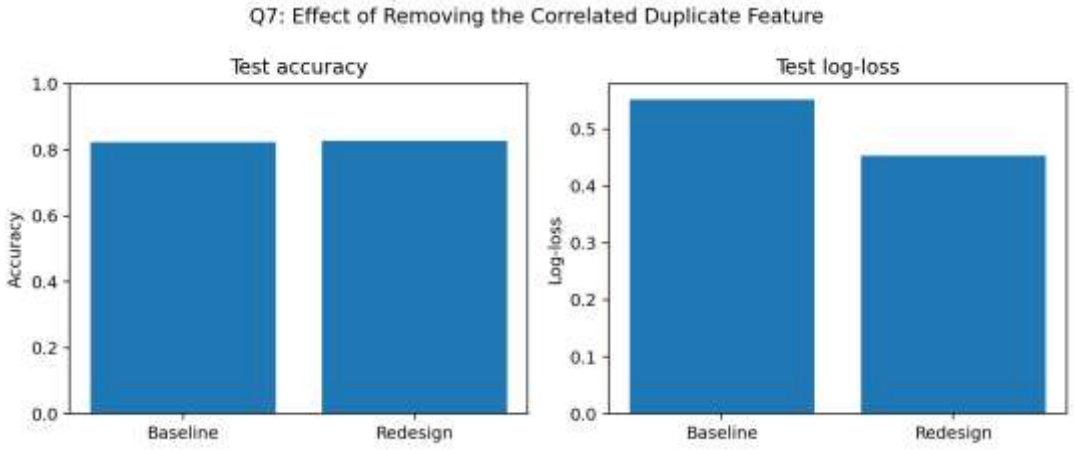


Figure 8. Q7 baseline and de-duplicated redesign comparison for accuracy and log-loss.

The redesign meets both pre-declared targets: log-loss improves by 17.81%, exceeding the 10% target, and accuracy increases by 0.50 percentage points rather than falling by more than 2 points. The result supports the diagnosis for this generated dataset.

F. Final design justification (2 marks)

The de-duplicated Naive Bayes model is preferred for this experiment because it improves probability quality and slightly improves accuracy without increasing model complexity. The original model may still be preferable when both correlated features carry distinct, complementary information or when removing one feature would discard useful measurements. In that case, a dependency-aware model such as TAN or a Bayesian belief network should be evaluated.

Remaining limitations include the synthetic nature of the data, one fixed random split and the use of one specific dependency strength. After deployment, monitor feature correlations, calibration/log-loss, accuracy, class-wise recall and changes in input distributions. Reassess the feature set when new sensors or measurements are introduced.

OVERALL CONCLUSION

Q1 showed how Bayes theorem updates a rare-fault prior and how Find-S and Candidate Elimination maintain or select consistent concepts. Q2 demonstrated the MDL trade-off between model complexity and training errors, and the difference between selecting a single MAP hypothesis and averaging over the posterior. Q3 implemented Naive Bayes and showed that correlated duplicate features can harm probability quality. Q4 represented explicit dependency using a Bayesian belief network and demonstrated explaining away. Q5 used EM to estimate hidden Gaussian mixture components and selected two components by BIC. Q6 compared PAC sample bounds, VC capacity and online mistake bounds. Q7 used an observed dependency problem to motivate and test a focused redesign.

The evidence supports a practical conclusion: the correct Bayesian or computational learning method depends on the structure of the data and the decision objective. Model assumptions, priors, smoothing, hidden variables, hypothesis-space capacity and computational cost all affect performance. A good design must be evaluated with the correct metric and its assumptions must be made explicit.

REPRODUCIBILITY AND SUBMISSION FILES

- Jupyter notebook: ITA06_Machine_Learning_Assignment_3_CO3_All_Codes_Updated.ipynb (all Q1-Q7 code and experiments; each question displays its own tables, plots and diagrams immediately after its calculations).
- Python source: assignment3_core.py (the same implementation used to generate the report results).
- Datasets: q3_loan_data.csv, q3_correlated_features_4000.csv (with train/test split labels), q4_bn_simulated_1000.csv, q5_gmm_500.csv, plus experiment result tables.
- Figures: all diagrams and experiment plots embedded in this report and included in the supporting ZIP.
- Results: assignment3_results.json and CSV files containing the full EM log-likelihood history, EM initialisations, CPT estimates, PAC bounds, simulation outputs and responsibilities.
- Random seeds: 4202 (Gibbs), 3303 (correlated Naive Bayes dataset), 4404 (Bayesian network simulation), 5505 (Gaussian mixture), 5600-5609 (EM initialisations), 6606/6616 (PAC experiments), and 6707 (online conjunction learner).

ACKNOWLEDGEMENT

Python libraries used: NumPy, pandas and Matplotlib for numerical computation, data handling and visualisation; scikit-learn CategoricalNB for independent verification only. The custom implementations required by the assignment are included in the notebook. AI assistance was used to help organise

explanations, code and report formatting. The student should run the notebook, review the derivations and be prepared to explain all calculations and results during the assessment verification discussion.

REFERENCES

- Mitchell, T. M. (1997). Machine Learning. McGraw-Hill. (Concept learning and Find-S.)
- Bishop, C. M. (2006). Pattern Recognition and Machine Learning. Springer. (Bayesian methods, mixture models and EM.)
- Koller, D., & Friedman, N. (2009). Probabilistic Graphical Models: Principles and Techniques. MIT Press. (Bayesian networks and inference.)
- Course document: ITA06 Machine Learning, Assignment 3 - CO3, supplied assignment brief.